PESQUISA EM RETRIEVAL-AUGMENTED GENERATION

RAG Atlas 200 DK

Benchmark de Otimização Pré-Query sobre a documentação técnica do Huawei Ascend 310 / Atlas 200 DK

Integrantes:
Murilo de Lima Barros
Otavio Antonio da Silva Junior
Ramon Vinicius Ferreira de Souza

JUSTIFICATIVA

Por Que a Documentação Huawei Ascend?

Alta especificidade de hardware

Especificações em nível de pino, arquitetura do DaVinci AI Core, memória LPDDR4X, formatos NC1HWC0 / FRACTAL_Z.

Tabelas densas de parâmetros

Flags do compilador ATC, configurações AIPP e matrizes de limites que quebram chunking ingênuo.

Sobreposição multi-framework

Operadores idênticos (Conv2D, BatchNorm) em Caffe e TensorFlow com restrições diferentes → risco de contaminação cruzada.

Estrutura hierárquica profunda

Árvores de cabeçalhos com 4 níveis: um chunk isolado sobre --precision_mode perde sentido sem o caminho pai.

Resultado: um teste de estresse ideal — se os métodos funcionam aqui, generalizam para documentação técnica em geral.

CONTEXTO

O Problema

- **Documentação técnica é densa —** especificações de hardware, tabelas de parâmetros e terminologia proprietária dificultam a recuperação precisa de informações.
- **LLMs isolados alucinam —** sem acesso ao contexto correto, modelos generativos produzem respostas plausíveis mas incorretas em domínios especializados.
- **A qualidade do RAG depende da query —** a etapa de retrieval é sensível à formulação da pergunta — pequenas variações mudam drasticamente os documentos recuperados.
- **Perguntas reais são conversacionais —** usuários fazem perguntas de acompanhamento com correferências ("e quanto ao formato de saída?") que quebram queries isoladas.

Pergunta central da pesquisa: como otimizar a query ANTES do retrieval melhora acurácia, reduz alucinação e aumenta a relevância?

VISÃO GERAL

Objetivo

Comparar quantitativamente 5 variantes de recuperação — do baseline simples à combinação das 3 técnicas de otimização pré-query — medindo acurácia das respostas e custo (chamadas de LLM) em um corpus técnico altamente especializado.

Baseline

Query original, sem nenhuma transformação

Step-Back

Abstração da pergunta para um conceito mais amplo antes do retrieval

Multi-Query / RAG-Fusion

Query reformulada em 3 variantes + fusão de rankings por RRF

Memória Conversacional

Reescrita da pergunta com o histórico do chat (perguntas de acompanhamento)

COMBINADO

Memória + Step-Back + Multi-Query leve com peso 2x para a query principal

VISÃO GERAL

Arquitetura do Pipeline RAG

1. INGESTÃO

PDF → Markdown
(docling / marker-pdf,
layout-aware)
+
pós-processamento

→

2. CHUNKING

recursivo
vs
por cabeçalhos
vs
atômico com tabelas

→

3. INDEXAÇÃO

Embeddings
(vector store Chroma)
+
baseline BM25/híbrido

→

4. PRÉ-QUERY ★

Step-Back

Multi-Query RRF

Memória
conversacional

→

5. GERAÇÃO

LLM com prompts
domain-specific
+
avaliação end-to-end

★ A etapa 4 é a contribuição principal da pesquisa — todos os experimentos são controlados contra o mesmo pipeline de base.

ROADMAP

Fase 1 — Ingestão e Chunking

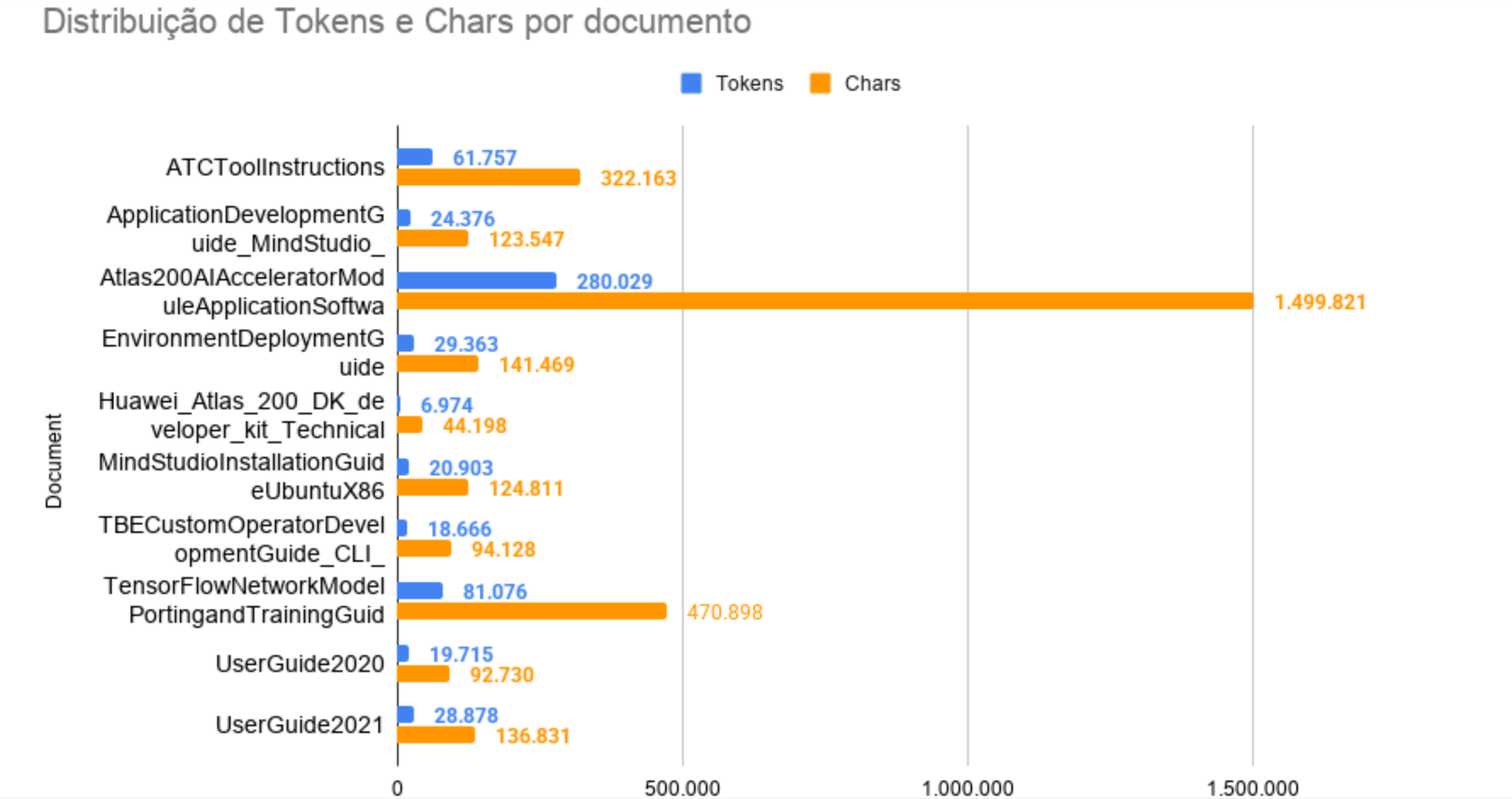
- **Extração —** PDF → Markdown com parsing layout-aware (docling)
- **Pós-processamento —** remoção de cabeçalhos/rodapés, reconexão de hífens técnicos, normalização de unidades
- **Validação —** estatísticas do corpus e checagem de integridade
- **Chunking —** comparação: recursivo vs. por cabeçalhos vs. atômico com preservação de tabelas

DIAGNÓSTICOS DE QUALIDADE

- Distribuição de tokens por chunk
- Auditoria de integridade de tabelas
- Legibilidade autônoma do chunk
- Exportação de payloads validados (JSON)

EXTRAÇÃO DE DADOS

O Corpus — 10 Manuais Huawei



NÚMEROS-CHAVE

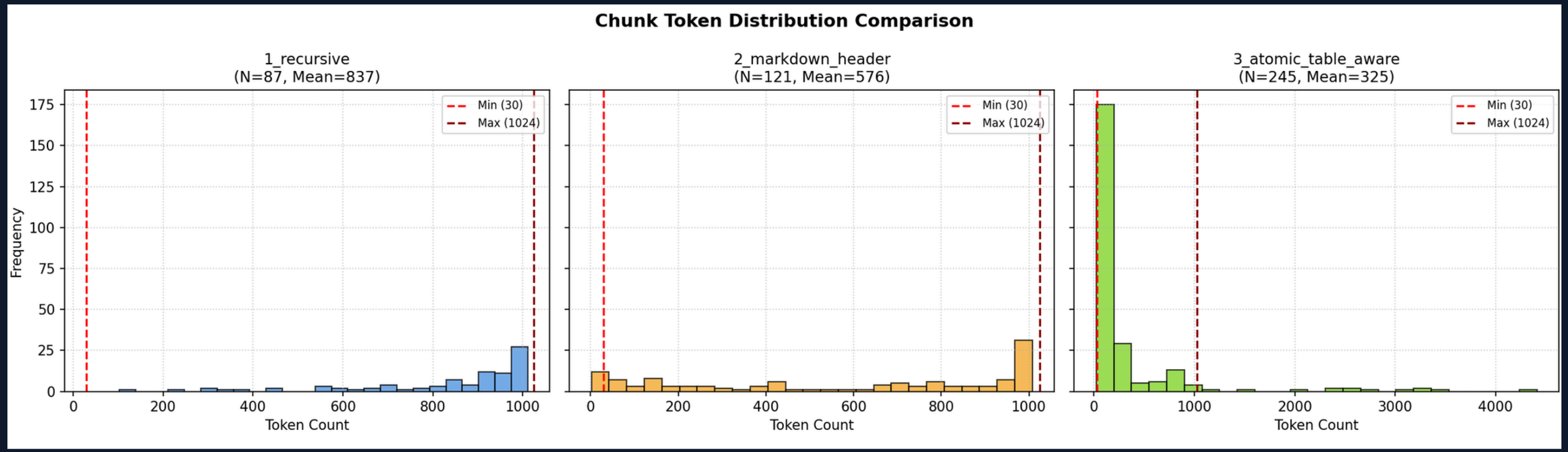
Markdown (Docling, TableFormer ACCURATE)

3.050.596 caracteres
571.737 tokens (cl100k_base)

715 tabelas detectadas

ESTRATÉGIAS

Comparação de Estratégias de Chunking



ROADMAP

Fases 2 e 3 — Indexação e Experimentos Pré-Query

FASE 2 — INDEXAÇÃO E BASELINE DE RETRIEVAL

- **Embeddings —** nomic-embed-text → transforma textos em vetores numéricos que representam características do texto.
- **Vector store —** ChromaDB → banco de dados vetorial
- **Métrica —** acurácia sobre perguntas validadas pelo Gemini Notebook

FASE 3 — EXPERIMENTOS PRÉ-QUERY (CONTRIBUIÇÃO PRINCIPAL)

- **Step-Back —** cadeia de abstração; ganho de recall em perguntas conceituais
- **Multi-Query / RAG-Fusion —** decomposição com LLM + re-ranqueamento RRF; ganhos de diversidade e cobertura
- **Memória conversacional —** janela deslizante vs. sumarização; resolução de correferências em multi-turno

ROADMAP

Fases 4 e 5 — Geração e Resultados

FASE 4 — GERAÇÃO E AVALIAÇÃO END-TO-END

- **Engenharia de prompts —** geração domain-specific com terminologia Ascend e especificações de operadores
- **Ajuste do LLM —** temperature, top-p, max tokens
- **Avaliação —** corretude, fidelidade ao contexto, acurácia de citações
- **Ablation —** isolar a contribuição de cada método pré-query vs. outros componentes do pipeline

FASE 5 — DOCUMENTAÇÃO E RESULTADOS

- **Relatório final —** benchmark com comparações quantitativas entre todos os métodos pré-query

RESULTADOS

5 Perguntas e Cada Resposta

1

What does the --input_fp16_nodes parameter do in the ATC tool?

Resposta retornada:

The `--input_fp16_nodes` parameter in the ATC tool is used to specify the name of an input node that has input data of type FP16 (half-precision floating point). This parameter is mutually exclusive with `--insert_op_conf`. When this parameter is set, it indicates that there is an input node in the original model that provides FP16 data. This is particularly useful when the input data to the model is expected to be in FP16 format, and the model needs to handle such inputs appropriately. Here are some key points about the `--input_fp16_nodes` parameter:

- **Mutual Exclusivity: It is mutually exclusive with `--insert_op_conf`, meaning you cannot use both parameters in the same ATC command.**
- **Usage Context: It is typically used in scenarios where the input data to the model is expected to be in FP16 format, and the model needs to be configured to handle this input type.**
- **Example Usage: You would specify the names of the input nodes that provide FP16 data, separated by semicolons.**

For example:
```sh
atc --model=HOME/test/resnet18.caffemodel --framework=0 --output=$HOME/test/out/caffe_resnet50 --input_fp16_nodes="data1;data2"
```

In summary, the `--input_fp16_nodes` parameter helps the ATC tool to correctly identify and handle input nodes that provide FP16 data, ensuring that the model conversion process accounts for this data type appropriately.

RESULTADOS

5 Perguntas e Cada Resposta

2

What are the main hardware differences between IT21DMDA and IT21VDMB board versions of the Atlas 200 DK?

Resposta retornada:

The main hardware differences between the IT21DMDA (old PCB) and IT21VDMB (new PCB) versions of the Atlas 200 DK are summarized as follows:

- **Power Input:**
  - IT21DMDA: Supports a power input range of 5 V to 28 V.
  - IT21VDMB: Uses a 12 V power input.

- **MIPI-CSI Port:**
  - IT21DMDA: Equipped with a 22-pin MIPI-CSI port without cables.
  - IT21VDMB: Features a 51-pin MIPI-CSI port with cables.

- **Number of GPIOs:**
  - IT21DMDA: Has 8 GPIOs.
  - IT21VDMB: Offers 11 GPIOs.

These differences highlight the enhanced capabilities and potential applications supported by the newer IT21VDMB version compared to the IT21DMDA.

RESULTADOS

5 Perguntas e Cada Resposta

3

What causes the error 'CheckPartitionSpace partition space check failed' during Atlas 200 DK driver upgrade?

Resposta retornada:

The error 'CheckPartitionSpace partition space check failed' during Atlas 200 DK driver upgrade typically indicates that there is insufficient unallocated space of 512 MB reserved on the SD card. This issue can be caused by using an old card creation script that did not allocate enough unallocated space.
To resolve this, you need to ensure that the SD card has at least 512 MB of unallocated space. If the space is insufficient, you can use tools like GParted to modify the partition size and allocate the required space.

RESULTADOS

5 Perguntas e Cada Resposta

4

How is the NC1HWC0 5D tensor format structured and what does C0 represent for FP16?

Resposta retornada:

The NC1HWC0 5D tensor format is structured as follows:

- **N: Batch size**
- **C1: Number of 16-element channels derived from the original C dimension**
- **H: Height**
- **W: Width**
- **C0: Size of the matrix computing unit in AI Core, which is equal to 16 for the FP16 data type.**

This means that the C dimension is divided by 16, and any remainder is padded with zeros to align with the C0 size. In summary, for FP16 data type, C0 is 16, and C1 is calculated as (C+15)//16, ensuring that the data is stored consecutively in memory.

5 Perguntas e Cada Resposta

5

Which CSC matrix coefficients are used in AIPP to convert YUV420SP_U8 to RGB888_U8 for JPEG images?



Resposta retornada:

For JPEG images, the CSC matrix coefficients used in AIPP to convert YUV420SP_U8 to RGB888_U8 are as follows:
For BT-601NARROW:
```
aipp_op { aipp_mode:
static input_format : YUV420SP_U8
csc_switch : true
rbuv_swap_switch : false
matrix_r0c0 : 256
matrix_r0c1 : 0
matrix_r0c2 : 359
matrix_r1c0 : 256
matrix_r1c1 : -88
matrix_r1c2 : -183
matrix_r2c0 : 256
matrix_r2c1 : 454
matrix_r2c2 : 0
input_bias_0 : 0
input_bias_1 : 128
input_bias_2 : 128 }
```

For BT-709NARROW:
```
aipp_op { aipp_mode:
static input_format :
YUV420SP_U8
csc_switch : true
rbuv_swap_switch : false
matrix_r0c0 : 298
matrix_r0c1 : 0
matrix_r0c2 : 409
matrix_r1c0 : 298
matrix_r1c1 : -100
matrix_r1c2 : -208
matrix_r2c0 : 298
matrix_r2c1 : 516
matrix_r2c2 : 0
input_bias_0 : 16
input_bias_1 : 128
input_bias_2 : 128 }
```

For BT-709NARROW (another variant):
```
aipp_op { aipp_mode: static input_format :
YUV420SP_U8
csc_switch : true
rbuv_swap_switch : false
matrix_r0c0 : 298
matrix_r0c1 : 0
matrix_r0c2 : 459
matrix_r1c0 : 298
matrix_r1c1 : -55
matrix_r1c2 : -136
matrix_r2c0 : 298
matrix_r2c1 : 541
matrix_r2c2 : 0
input_bias_0 : 16
input_bias_1 : 128
input_bias_2 : 128 }
```

5 Perguntas e Cada Resposta

Resposta retornada:

For JPEG images, the CSC matrix coefficients used in AIPP to convert YUV420SP_U8 to RGB888_U8 are as follows:



For BT-601NARROW:

```
aipp_op { aipp_mode: static input_format : YUV420SP_U8
csc_switch : true
rbuv_swap_switch : false
matrix_r0c0 : 256
matrix_r0c1 : 0
matrix_r0c2 : 359
matrix_r1c0 : 256
matrix_r1c1 : -88
matrix_r1c2 : -183
matrix_r2c0 : 256
matrix_r2c1 : 454
matrix_r2c2 : 0
input_bias_0 : 0
input_bias_1 : 128
input_bias_2 : 128 }
```

For BT-709NARROW:

```
aipp_op { aipp_mode: static input_format : YUV420SP_U8
csc_switch : true
rbuv_swap_switch : false
matrix_r0c0 : 298
matrix_r0c1 : 0
matrix_r0c2 : 409
matrix_r1c0 : 298
matrix_r1c1 : -100
matrix_r1c2 : -208
matrix_r2c0 : 298
matrix_r2c1 : 516
matrix_r2c2 : 0
input_bias_0 : 16
input_bias_1 : 128
input_bias_2 : 128 }
```

For BT-709NARROW (another variant):

```
aipp_op { aipp_mode: static input_format : YUV420SP_U8
csc_switch : true
rbuv_swap_switch : false
matrix_r0c0 : 298
matrix_r0c1 : 0
matrix_r0c2 : 459
matrix_r1c0 : 298
matrix_r1c1 : -55
matrix_r1c2 : -136
matrix_r2c0 : 298
matrix_r2c1 : 541
matrix_r2c2 : 0
input_bias_0 : 16
input_bias_1 : 128
input_bias_2 : 128 }
```

The answer includes the correct CSC matrix coefficients for JPEG images but incorrectly includes additional variants for BT-709NARROW that contradict the Gemini Notebook answer.

5 Perguntas e Cada Resposta

Resposta retornada:

For JPEG images, the CSC matrix coefficients used in AIPP to convert YUV420SP_U8 to RGB888_U8 are as follows:

For BT-601NARROW:
```
aipp_op { aipp_mode: static input_format : YUV420SP_U8 csc_switch : true rbuv_swap_switch : false matrix_r0c0 : 256 matrix_r0c1 : 0
...
```
For BT-709NARROW:
```
...
matrix_r0c0 : 298 matrix_r0c1 : 0
...
```

Table 4-1 YUV420SP_U8 to RGB888_U8

JPEG	BT-601NARROW	BT-709NARROW
aipp_op { aipp_mode: static input_format : YUV420SP_U8 csc_switch : true rbuv_swap_switch : false matrix_r0c0 : 256 matrix_r0c1 : 0 matrix_r0c2 : 359 matrix_r1c0 : 256 matrix_r1c1 : -88 matrix_r1c2 : -183 matrix_r2c0 : 256 matrix_r2c1 : 454 matrix_r2c2 : 0 input_bias_0 : 0 input_bias_1 : 128 input_bias_2 : 128 }	aipp_op { aipp_mode: static input_format : YUV420SP_U8 csc_switch : true rbuv_swap_switch : false matrix_r0c0 : 298 matrix_r0c1 : 0 matrix_r0c2 : 409 matrix_r1c0 : 298 matrix_r1c1 : -100 matrix_r1c2 : -208 matrix_r2c0 : 298 matrix_r2c1 : 516 matrix_r2c2 : 0 input_bias_0 : 16 input_bias_1 : 128 input_bias_2 : 128 }	aipp_op { aipp_mode: static input_format : YUV420SP_U8 csc_switch : true rbuv_swap_switch : false matrix_r0c0 : 298 matrix_r0c1 : 0 matrix_r0c2 : 459 matrix_r1c0 : 298 matrix_r1c1 : -55 matrix_r1c2 : -136 matrix_r2c0 : 298 matrix_r2c1 : 541 matrix_r2c2 : 0 input_bias_0 : 16 input_bias_1 : 128 input_bias_2 : 128 }

For BT-709NARROW (another variant):
```
...
matrix_r0c0 : 298
matrix_r0c1 : 0
matrix_r0c2 : 459
matrix_r1c0 : 298
```

Resultados

Acurácia

VARIAÇÃO	ACC	CALLS (média)	TEMPO (20 questões)
BASELINE	80%	1	54s
STEPBACK	70%	2	49s
MULTIQUERY	75%	2	58s
MEMORY	75%	2	51s
COMBINED	90%	3.1	57s

```
# ============== PARAMETERS CONFIGURATION ==============
K                   = 4     # chunks entregues ao gerador
N_RUNS              = 1     # execuções por variante
SEED_BASE           = 42    # seed para reprodutibilidade
GEN_TEMP            = 0.5   # temperatura da resposta final
REWRITE_TEMP  = 0.3   # temperatura das chamadas de reescrita
N_VARIANTS          = 3     # reformulações no Multi-Query
RRF_C               = 15    # constante do Reciprocal Rank Fusion
MEMORY_DISTRACTOR = True
FORCE               = True
# =====================================================
```

Obrigado!

github.com/ramonvfs/RAG_Atlas200DK
Benchmark de otimização pré-query em corpus técnico real